Keywords: continual learning • retrieval-augmented agents • tutorial video understanding • nonparametric adaptation

Can LLM Agents Learn by Watching Tutorial Videos?

Tutorial Agentic Learning Turns Video Procedures into Frozen-LLM Memory with No Weight Updates

By Haoyu Zhao, Zafeirios Fountas, Jun Wang

arXiv:2611.01460 • cs.LG / cs.AI / cs.CL / cs.CV • Submitted 1 Nov 2026

Tutorial Agentic Learning (TAL) enables a frozen large language model to acquire procedural knowledge from instructional videos at inference time, without any weight updates. Tested on VidLearn—a new multimodal benchmark of 72 chess tutorial videos spanning 12 openings—TAL performance scales with the number of videos consumed, demonstrating genuine procedural learning from visual and language content.

AT A GLANCE

Problem: LLM agents require expensive retraining to acquire new procedural skills, blocking continual learning at deployment.

Why it matters: Tutorial videos are abundant and contain rich procedural knowledge; efficiently converting them to agent capability would eliminate per-skill fine-tuning cycles.

Method: Videos are segmented into timestamped procedural entries and stored in a retrieval-augmented memory bank; at inference time, top-k entries provide in-context guidance to a frozen LLM.

Result: TAL outperforms the same LLM without video context; accuracy scales with number of videos ingested on the VidLearn benchmark.

Evidence: VidLearn: 72 chess tutorial videos, 12 openings, multimodal QA; dense retrieval ablations over retrieval count k.

The Big Picture

The boundary between training and inference has been treated as fixed in standard machine learning: an agent learns during training and exploits what it learned during deployment. But in real-world settings, new procedural knowledge becomes available continuously—software tutorials, instructional videos, how-to guides—and re-initiating a training run for each new skill is impractical.

Tutorial Agentic Learning (TAL) reframes this problem: rather than modifying weights, it treats instructional videos as a persistent, queryable memory that augments a frozen agent at inference time. The approach is inspired by retrieval-augmented generation (RAG) for factual QA, but extends the paradigm to procedural, visually grounded knowledge embedded in sequential video content.

Why Existing Approaches Fall Short

Fine-tuning on instructional data requires labeled examples and gradient computation, is prone to catastrophic forgetting of previously learned skills, and must be repeated for each new skill domain.

In-context learning from text summaries loses the visual content of videos and fails to capture the sequential, state-action structure of procedural knowledge, particularly in domains like chess or software manipulation where board/screen state is essential.

Vision-language models applied to full video at inference time are computationally prohibitive: processing 72 tutorial videos at every query step would require multiple full video passes per query.

Core Mechanism

TAL operates in two phases. During **offline indexing**, each video is segmented into timestamped steps using a vision-language model (VLM). Each step is converted into a structured procedural entry:

$$e_i = (\text{state}_i,\ \text{action}_i,\ \text{rationale}_i)$$

where state_i is a visual embedding of the board/screen, action_i is the described move or operation, and rationale_i is the narrated explanation. All entries are stored in a FAISS-indexed memory bank.

During **inference-time retrieval**, given a query q, the top-k entries are retrieved via dense similarity search:

$$\mathcal{E}^* = \arg\top_k \{\cos(\phi(q),\ \phi(e_i))\}_{i=1}^{N}$$

where $\phi(\cdot)$ is a frozen CLIP encoder. The retrieved entries are formatted as structured in-context examples and provided to the frozen LLM for final answer generation.

The VidLearn Benchmark

VidLearn is a new multimodal QA benchmark constructed from 72 publicly available chess tutorial videos spanning 12 chess openings (Sicilian Defense, King's Indian, French Defense, and 9 others). Each video is annotated with key board positions, move-by-move explanations, and multiple-choice and free-form questions. The benchmark tests whether an agent can correctly identify optimal continuations from visually presented board states, drawing on knowledge explicitly taught in the videos.

The 12 openings are partitioned into train (8) and test (4) splits to evaluate cross-opening transfer.

Learning or Inference Procedure

Offline (once per video set):

1. Segment each video into T_v timestamped steps using a VLM.
2. Embed each step as a (state, action, rationale) triple.
3. Encode each triple with CLIP and add to the FAISS index.

Inference (per query):

1. Encode the query board image and question text with CLIP.
2. Retrieve top-k entries from the memory bank.
3. Format as in-context examples and pass to the frozen LLM.
4. Return the LLM's generated answer.

No gradient computation occurs at any stage of inference.

What the Guarantee Says

TAL provides no formal learning-theoretic guarantee. The empirical claim is a scaling law: accuracy on VidLearn test questions increases as the number of ingested training-opening videos grows, leveling off around 50 videos per opening. This suggests that the memory bank successfully captures the unique procedural patterns in a given opening up to a saturation point. No guarantee is provided for out-of-distribution openings, though cross-opening transfer is observed for structurally related sub-variants.

Experimental Findings

- **Accuracy vs. baseline:** TAL with 72 videos achieves substantially higher accuracy than the same LLM without video context on VidLearn multiple-choice questions.
- **Scaling:** Accuracy increases monotonically with the number of ingested videos, with diminishing returns above 50 videos per opening.
- **Cross-opening transfer:** Positive transfer is observed for Sicilian Defense sub-variants, with 4.1% accuracy improvement on unseen variations when related-opening videos are present.
- **Retrieval ablation:** Performance improves with k up to $k = 5$, then plateaus, suggesting diminishing returns from additional in-context examples beyond 5.

Ablations and Interpretation

Without visual state embeddings: Replacing CLIP visual embeddings with text-only embeddings of move descriptions reduces accuracy by 7.3%, confirming that the board-state visual signal is essential for precise retrieval.

Without the rationale field: Removing the rationale from in-context entries reduces accuracy by 4.9%, indicating that the LLM uses explanatory text from videos to reason about move motivations.

Approximate vs. exact search: Using exact L2 search instead of approximate FAISS search improves accuracy by less than 0.5%, confirming that approximate retrieval is sufficient and scalable.

Impact of VLM quality for segmentation: Replacing the full VLM with a smaller captioning model for step segmentation reduces accuracy by 6.1%, highlighting that the quality of initial video parsing matters.

Reference

Haoyu Zhao, Zafeirios Fountas, Jun Wang. **Can Large Language Model Agents Learn by Watching Tutorial Videos?** arXiv:2611.01460. 2026. https://arxiv.org/abs/2611.01460

Keywords: zeroth-order optimization • reinforcement learning for LLMs • noise amplification • memory-efficient fine-tuning

The Cost of Mismatch

Noise Amplification in Zeroth-Order Reinforcement Learning Scales as $1/\mu^2$; Matched-Data BOLT Cancels It

By Lianmin Chen, Junbin Qiu, Chenxing Wei, Yao Shu, Kun He

arXiv:2611.03260 • cs.LG • NeurIPS 2026 Poster

Zeroth-order reinforcement learning for large language models suffers from a fundamental noise amplification effect in the mismatched-data setting: gradient variance scales as $1/\mu^2$, creating an irreducible noise floor as the perturbation scale μ shrinks. The authors identify the matched-data BOLT estimator as the antidote, derive ZO-BOLT, and demonstrate 3–5× lower gradient variance and 40% faster convergence on GSM8K and HumanEval.

AT A GLANCE

Problem: ZO RL for LLMs is unstable in practice; the reason is not understood theoretically.

Why it matters: ZO methods are the only approach that enables policy-gradient RL without backpropagation, keeping memory at inference cost; instability blocks their practical use.

Method: The authors prove a $1/\mu^2$ variance lower bound for mismatched-data ZO RL and derive ZO-BOLT, which shares rollout randomness between both perturbation evaluations to cancel the dominant noise term.

Result: ZO-BOLT reduces gradient variance by 3–5× and reduces forward passes to target accuracy by 40% on GSM8K and HumanEval.

Evidence: Gradient variance measurements; convergence curves on mathematical reasoning (GSM8K, MATH) and code generation (HumanEval).

The Big Picture

Policy gradient RL requires differentiating through a reward signal that is typically non-differentiable (discrete text actions, binary correctness signals). Zeroth-order methods estimate the gradient by measuring reward differences between two parameter-perturbed policies, avoiding backpropagation entirely. This makes ZO RL attractive for fine-tuning LLMs with RL, since the memory footprint equals that of inference.

Despite this appeal, practitioners have found ZO RL for LLMs to be unstable—high variance, slow convergence, sensitive to hyperparameters. In contrast, ZO supervised fine-tuning methods such as MeZO work reliably. This discrepancy motivates the central question of the paper: is there a fundamental difference between ZO SFT and ZO RL that explains the instability?

Why Existing Approaches Fall Short

Standard ZO RL uses the two-point estimator:

$$\hat{g} = \frac{r(\theta + \mu z) - r(\theta - \mu z)}{2\mu} \cdot z$$

where the two reward evaluations use independently sampled rollouts. The mismatch between these rollouts is the fundamental problem.

Smaller perturbation scale μ reduces the bias of the estimator but, as the paper shows, the variance grows proportionally to $1/\mu^2$—the two effects cannot be balanced simultaneously.

More rollouts per update reduces variance linearly but does not address the $1/\mu^2$ structure, requiring a quadratic increase in rollouts to halve the variance floor.

Core Mechanism: The $1/\mu^2$ Lower Bound

Let $r(\theta)$ be the stochastic reward with noise variance σ^2_{env}. The paper shows that the variance of the standard ZO RL estimator decomposes as:

$$\text{Var}(\hat{g}) = \frac{\sigma^2_{\text{env}}}{2\mu^2} + O\left(\frac{\sigma^4_{\text{env}}}{\mu^4}\right) + \text{signal terms}$$

The leading term $\sigma^2_{\text{env}}/(2\mu^2)$ grows without bound as $\mu \to 0$, independent of sample size. This is the *mismatch penalty*: it arises specifically because the two evaluations $r(\theta+\mu z)$ and $r(\theta-\mu z)$ use different rollout seeds, so their noise does not cancel in the difference.

The BOLT Solution

Boltzmann Targeted SFT (BOLT) in the matched-data setting shares the same rollout seed for both evaluations. Under matched randomness:

$$\tilde{g}_{\text{BOLT}} = \frac{r_s(\theta + \mu z) - r_s(\theta - \mu z)}{2\mu} \cdot z$$

where both evaluations use rollout seed s. The noise now cancels:

$$r_s(\theta + \mu z) - r_s(\theta - \mu z) = \nabla_\theta r(\theta)^\top (2\mu z) + O(\mu^3)$$

with no leading noise term. The variance reduces to:

$$\text{Var}(\tilde{g}_{\text{BOLT}}) = O(1)$$

independent of μ, eliminating the $1/\mu^2$ amplification entirely.

ZO-BOLT Algorithm

ZO-BOLT instantiates matched-data perturbation for LLM RL fine-tuning:

1. Sample a random direction z in parameter space.
2. Fix a rollout seed s.
3. Evaluate the model under $\theta+\mu z$ with seed s to obtain reward r^+.
4. Evaluate the model under $\theta-\mu z$ with the same seed s to obtain reward r^-.
5. Compute gradient estimate $\hat{g} = (r^+ - r^-)/(2\mu) \cdot z$.
6. Update: $\theta \leftarrow \theta - \eta\hat{g}$.

The matched-seed constraint adds zero computational overhead—it requires only fixing the random seed before each rollout pair.

What the Guarantee Says

Theorem 1 (informal): For any ZO estimator in the mismatched-data setting with environment noise $\sigma^2_{\text{env}} > 0$, the gradient variance is at least $\sigma^2_{\text{env}}/(2\mu^2)$ for perturbation scale μ. No mismatched estimator can circumvent this bound.

Theorem 2 (informal): The ZO-BOLT matched-data estimator achieves $O(1)$ variance, independent of μ. Crucially, this requires only that $r_s(\cdot)$ is deterministic given the seed, which holds for any pseudorandom rollout procedure.

The bounds are non-asymptotic and hold for finite d and $\mu > 0$.

Experimental Findings

- **Gradient variance (GSM8K):** ZO-BOLT reduces gradient variance by 3.2× over standard ZO RL at $\mu = 10^{-3}$, and 5.1× at $\mu = 10^{-4}$.
- **Convergence (GSM8K):** ZO-BOLT reaches 70% accuracy with 40% fewer forward passes than standard ZO RL.
- **Convergence (HumanEval):** ZO-BOLT reaches 50% Pass@1 with 35% fewer forward passes.
- **MATH:** ZO-BOLT final accuracy improves by 4.3% over standard ZO RL with the same forward-pass budget.

Ablations and Interpretation

Effect of μ on ZO-BOLT: Unlike standard ZO RL, ZO-BOLT performance is insensitive to μ across the range 10^{-4}–10^{-2}, confirming that the mismatch penalty has been eliminated.

Seed granularity: The paper evaluates fixing the seed per rollout pair (default) vs. per batch. Per-pair seeding gives lower variance, as expected from the theory.

Comparison to first-order RL: ZO-BOLT is 12× slower to converge than GRPO (a first-order RL method) on GSM8K, but uses 8× less memory—a favorable trade-off when GPU memory is the bottleneck.

Reference

Lianmin Chen, Junbin Qiu, Chenxing Wei, Yao Shu, Kun He. **The Cost of Mismatch: Noise Amplification in Zeroth-Order Reinforcement Learning.** arXiv:2611.03260. NeurIPS 2026. https://arxiv.org/abs/2611.03260

Keywords: Transformer theory • attention low-rank structure • random matrix theory • softmax spectral analysis

Spectral Degeneration of Self-Attention

Under Isotropic Score Geometry, Softmax Collapses the Attention Spectrum; Causal Masking Grows Stable Rank as $O(\log L)$

By Mengda Li, Jianfeng Yao

arXiv:2611.02778 • cs.LG / stat.ML • NeurIPS 2026 • CUHK Shenzhen

An idealized spectral model of self-attention under isotropic score geometry reveals why attention matrices are approximately low-rank: the softmax nonlinearity amplifies dominant modes while collapsing the bulk of the spectrum to zero. For bidirectional attention, the stable rank stays bounded as sequence length grows; for causal attention, it grows logarithmically, implying that decoder-only models are intrinsically less compressible per sequence position than encoders.

AT A GLANCE

Problem: Attention matrices are empirically low-rank, but the theoretical reason for this degeneration is unknown.

Why it matters: Understanding why attention collapses informs principled low-rank compression, regularization design, and the choice between encoder and decoder architectures.

Method: Idealized Gaussian random matrix model for the score matrix, combined with free probability theory to track how softmax distorts the eigenspectrum.

Result: Softmax amplifies dominant eigenvalues while collapsing the bulk; stable rank is $O(1)$ for bidirectional and $O(\log L)$ for causal attention.

Evidence: Analytical derivations validated on pre-trained GPT-2 and BERT attention heads and synthetic random attention matrices.

The Big Picture

Empirical studies of trained Transformers consistently report that attention matrices are approximately low-rank: a small number of singular values carry most of the mass, while the remaining singular values are near zero. Methods such as Linformer, Nystromformer, and FlashAttention-low-rank exploit this empirically, but without theoretical justification for when and why it holds.

Is low-rank structure a consequence of training dynamics, or is it intrinsic to the softmax attention computation? This paper answers the second question affirmatively: even without training, the softmax nonlinearity induces spectral degeneration in the attention matrix when the pre-softmax score matrix has isotropic geometry.

Why Existing Theory Falls Short

Empirical studies document low-rank structure but do not explain its origin; they attribute it to gradient dynamics without analysing the effect of the softmax function itself.

Linear attention theory replaces softmax with a kernel approximation and analyzes the resulting linear map, but this fundamentally changes the operation and does not explain the behavior of softmax attention.

Rank-collapse results from overparameterized network theory apply to weight matrices in MLP layers and do not transfer to the compositional query–key product structure of attention.

The Isotropic Score Model

Let $S \in \mathbb{R}^{L\times L}$ be the pre-softmax score matrix with $S_{ij} = q_i^\top k_j/\sqrt{d}$. The isotropic assumption models S as a Wigner-type random matrix:

$$S = \frac{1}{\sqrt{L}} W + \lambda \mathbf{u}\mathbf{v}^\top$$

where W has i.i.d. entries with mean 0 and variance 1, λ is the signal strength, and $\mathbf{u}, \mathbf{v}$ represent a planted dominant mode. This models a "generic" attention head in which most query–key pairs are unrelated (W term) but a few share a strong structural alignment (λ term).

Main Theoretical Results

Softmax Spectral Transformation. Let $A = \mathrm{softmax}(S)$. Under the isotropic model, in the limit $L \to \infty$ with fixed d, the empirical spectral distribution of A converges to a law concentrated near zero, with an isolated outlier eigenvalue at:

$$\lambda_1(A) \approx \frac{e^{\lambda/\sqrt{L}}}{\mathbf{1}^\top e^{S\mathbf{v}}/L}$$

while the bulk of the spectrum satisfies $\lambda_i(A) = O(1/L)$ for $i \geq 2$. The **stable rank** $\rho(A) = \|A\|_F^2/\|A\|_2^2$ therefore obeys:

$$\rho(A) = O(1) \quad \text{(bidirectional attention)}$$

Causal Attention. The causal mask introduces a triangular structure in S: row i can only attend to columns $j \leq i$. The row-normalization of softmax then has a

gradient: earlier rows normalize over fewer tokens. The analysis shows that this asymmetry causes the stable rank to grow as:

$$\rho(A_{\text{causal}}) = O(\log L)$$

Proof Overview

The derivation proceeds in three steps:

1. Apply the semicircle law from random matrix theory to characterize the eigenspectrum of S.
2. Use free probability calculus to track the distortion of the spectral measure through the entrywise exponential $\exp(S)$.
3. Show that the row-normalization step of softmax is approximated by a rank-1 update to $\exp(S)$ in the high-dimensional limit, and derive its effect on singular values.

For causal attention, the proof additionally shows that the per-row normalization constants form a monotonically increasing sequence in the row index, leading to a non-uniform reweighting of singular vectors that grows the stable rank logarithmically.

What the Guarantee Says

The spectral collapse results are asymptotic ($L \to \infty$, fixed d) and require the isotropic geometry assumption, which may not hold for specialized attention heads in trained models. The paper validates that the predictions remain qualitatively accurate for finite L (up to $L = 2048$) and for trained heads in GPT-2 and BERT, but provides no non-asymptotic concentration bounds. The results apply to the attention matrix A itself, not to the final representations AV, which may have different rank properties depending on the value matrix V.

Empirical Validation

- **Synthetic:** Stable rank under the isotropic model matches analytical predictions at $L \in \{128, 512, 2048\}$.
- **GPT-2 (causal):** Empirical stable rank grows as $1.3 \log L + 0.7$ across heads, consistent with the $O(\log L)$ prediction.
- **BERT (bidirectional):** Empirical stable rank stays in the range $[1.8, 4.2]$ across heads and all L tested, consistent with the $O(1)$ bound.
- **Head specialization:** The planted outlier λ correctly predicts the magnitude of the dominant singular value across heads with varying degrees of specialization.

Implications for Practice

Low-rank compression: The $O(1)$ stable rank for bidirectional attention justifies using rank-r approximations with r independent of L. For causal models, rank should scale mildly with L.

Architecture choice: Encoder models can achieve the same compression ratio regardless of context length. Decoder models become progressively harder to compress at long contexts.

Regularization: Encouraging isotropic score geometry during training via spectral normalization of Q and K projections may stabilize the spectral structure and improve compressibility.

Reference

Mengda Li, Jianfeng Yao. **Spectral Degeneration of Self-Attention under Isotropic Score Geometry.** arXiv:2611.02778. NeurIPS 2026. https://arxiv.org/abs/2611.02778

Keywords: multi-agent reasoning • strategy optimization • meta-learning • agentic AI

Strategist: Designing Agentic Reasoning at Scale

A Meta-Agent Builds a Growing Library of Executable Strategy Blueprints, Winning 24 of 30 Benchmarks at 48% Lower Cost

By Taiming Lu, Qi Xu, Zhigang Hua, Zhuang Liu

arXiv:2611.03618 • cs.AI / cs.CL • NeurIPS 2026 Poster

Strategist is a meta-agent that automatically designs inference strategies for new tasks by representing each strategy as executable Python code, building a reusable library of blueprints that grows across tasks. Evaluated on 30 reasoning benchmarks, Strategist wins 24 and achieves 8.5 absolute percentage points higher average accuracy than the strongest fixed-strategy baseline at 48% lower inference cost. Strategies transfer across model backends the meta-agent never optimized on.

AT A GLANCE

Problem: Fixed pipeline templates (chain-of-thought, tree-of-thought, debate) do not generalize across domains and require manual design.

Why it matters: The topology of multi-agent reasoning is a critical variable for performance; automating its design could yield consistent improvements across tasks without human effort.

Method: Strategies are Python programs (blueprints); a meta-agent retrieves related blueprints, proposes candidates, evaluates them on a dev set, revises them, and stores all valid candidates in a library.

Result: 8.5pp accuracy improvement and 48% cost reduction over the best fixed strategy, on 24 of 30 benchmarks.

Evidence: 30 benchmarks spanning math, coding, science, and commonsense; transfer experiments to Gemini-2.5 backend.

The Big Picture

Multi-agent systems for reasoning typically rely on templates: chain-of-thought prompts the model to reason step-by-step, tree-of-thought expands multiple reasoning branches, and debate pits agents against one another. These structures are designed by researchers for specific task types and do not automatically adapt to new domains. Meanwhile, automated prompt engineering optimizes over discrete tokens but leaves the structural topology of the reasoning process fixed.

The key insight of Strategist is that the topology itself—how agents are orchestrated, how outputs are routed and verified—is a programmable object that can be optimized. Representing strategies as Python programs makes them composable, executable, and amenable to code-level search by a language model.

Why Existing Approaches Fall Short

Fixed templates (chain-of-thought, majority voting) are designed for average-case performance across broad task categories. They cannot capture the structural insights that are most effective for a specific task type, such as using a symbolic calculator for arithmetic sub-steps or running parallel hypotheses for spatial reasoning.

Prompt optimization methods (AutoPrompt, DSPy, APE) treat the strategy as a fixed function and optimize over its textual inputs, but do not change the number of agents, the verification steps, or the routing logic. They are limited to the structural assumptions baked into the initial template.

Human-designed multi-agent systems require expert knowledge and significant engineering time, and the resulting designs are rarely reusable across domains.

Core Mechanism: Strategy as Code

Each strategy in Strategist is a Python program, or *blueprint*, that takes a task description and input as arguments and returns an answer by orchestrating one or more LLM calls. A blueprint may:

- Decompose the problem into typed sub-problems and dispatch each to a specialized agent.
- Route outputs conditionally based on model confidence or consistency checks.
- Call helper functions from other library blueprints as sub-routines.
- Aggregate results via voting, scoring, or symbolic verification.

This representation makes strategies fully executable and inspectable, enables compositional reuse, and gives the meta-agent a structured search space that is richer than prompt tokens alone.

The Design Loop

Given a new task T and a small development set $\mathcal{D}$:

1. **Retrieve:** Encode T with a frozen text encoder and retrieve top-k blueprints from the library by cosine similarity.
2. **Propose:** The meta-agent generates 3–5 candidate blueprints as Python code, guided by retrieved examples and the target task description.

3. **Evaluate:** Execute each candidate on $\mathcal{D}$; record accuracy a_i and total token cost c_i.
4. **Revise:** The meta-agent reviews evaluation results and produces a final revised blueprint for T.
5. **Store:** All candidates and the final blueprint are added to the library with metadata $(a_i, c_i, \text{task-type tags})$.

The loop runs for 3 rounds per task, with early stopping if the best candidate exceeds a target accuracy of $\alpha^* = 0.9$.

The Strategy Library

The library is initialized with 20 hand-designed baselines. After processing all 30 benchmarks in order, the library contains ∼150 blueprints. The compositional reuse mechanism means that blueprints are not monolithic: a later task (e.g., GPQA) can invoke the "verify-arithmetic" subroutine discovered for GSM8K and the "multi-perspective synthesis" routine from debate-style blueprints.

What the Guarantee Says

Strategist provides no PAC-learning or regret guarantee. The empirical claim is competitive: on the 30-benchmark suite, the designed strategies outperform all fixed baselines (including the best oracle fixed strategy per task). No theoretical bound on the number of meta-agent iterations needed to find a good strategy is given. The transfer result (Section 7) is empirical and suggests cross-model generality, but the mechanism is not formally analyzed.

Experimental Findings

- **Accuracy (30 benchmarks):** Strategist wins 24 of 30; average improvement is **8.5pp** over the strongest fixed baseline (which is the oracle best-fixed-strategy per task).
- **Cost:** Despite higher quality, Strategist uses **48% fewer tokens** on average, because it avoids unnecessary LLM calls and uses cheaper model tiers for sub-tasks.
- **Library growth:** Average accuracy on a held-out set of 5 benchmarks improves by 3.1pp after processing the first 25 tasks, confirming compositional library benefit.
- **Transfer to Gemini-2.5:** Strategies discovered with GPT-6 improve Gemini-2.5 chain-of-thought accuracy by **6.1pp** on average, confirming cross-model transferability.

Ablations and Interpretation

Without retrieval: Removing library retrieval (proposing strategies from scratch) reduces accuracy by 4.4pp, confirming that the library enables meaningful compositional reuse.

Strategies as text vs. code: Replacing Python blueprints with natural-language strategy descriptions reduces accuracy by 2.9pp, as natural-language strategies are less precise and cannot directly invoke verified sub-routines.

Without iterative revision: Single-round (no revision step) performance is 3.7pp below the full 3-round loop, with diminishing returns in rounds 2 and 3.

Meta-agent backbone: Replacing GPT-6 as the meta-agent with GPT-5 reduces strategy quality by 2.1pp on average, but the benefit of strategy design over no design is maintained across all backbones tested.

Reference

Taiming Lu, Qi Xu, Zhigang Hua, Zhuang Liu. **Strategist: Designing Agentic Reasoning at Scale.** arXiv:2611.03618. NeurIPS 2026. https://arxiv.org/abs/2611.03618

Keywords: zeroth-order optimization • memory-efficient fine-tuning • variational multiscale methods • LLM adaptation

ENGINE: Variational Multi-Scale Zeroth-Order LLM Fine-Tuning

A Closed-Form Subgrid Correction from Computational Mechanics Yields 2.61× Faster ZO Convergence at Zero Memory Overhead

By Zhuoli Ouyang, Bowen Li, Chen Hao, Yejin Choi

arXiv:2611.03293 • cs.LG • Submitted 2 Nov 2026

ENGINE applies the Variational MultiScale (VMS) decomposition from computational fluid mechanics to zeroth-order LLM fine-tuning. By decomposing perturbations into a low-rank coarse component and computing a closed-form subgrid correction that reintroduces unresolved curvature at no additional cost, ENGINE achieves a 2.61× speedup in forward passes over prior ZO methods with zero additional peak memory, and consistently outperforms state-of-the-art ZO baselines on NLP benchmarks.

AT A GLANCE

Problem: Zeroth-order optimization for LLM fine-tuning is slow to converge because low-rank perturbation subspaces miss curvature from the complement space.

Why it matters: ZO fine-tuning is the only way to train LLMs with RL rewards or in memory-limited settings without backpropagation; improving its efficiency makes it practical.

Method: VMS decomposes the parameter space into a low-rank coarse component (updated with 2 forward passes) and a fine-scale complement; a closed-form subgrid correction reintroduces complement curvature with no extra forward passes.

Result: 2.61× speedup in forward passes over LOREN; zero additional peak memory overhead; consistent improvements on SuperGLUE, MMLU, and instruction-following.

Evidence: GPT-J-6B on SuperGLUE, Llama-3-8B on MMLU, and instruction-following on Alpaca Eval.

The Big Picture

Zeroth-order (ZO) optimization enables fine-tuning LLMs without backpropagation, reducing memory from $O(\text{parameters})$ to $O(1)$ for the optimizer state. This matters in three settings: (1) fine-tuning on GPUs with insufficient VRAM for gradients, (2) fine-tuning through black-box API access where gradients are unavailable, and (3) fine-tuning with non-differentiable reward signals such as human feedback or unit-test pass rates.

The main limitation is convergence speed: estimating gradients by measuring reward differences between perturbed parameters is noisy. Restricting perturbations to a low-rank subspace (as in MeZO and LOREN) reduces noise but introduces a systematic bias: curvature from the complement of the subspace is ignored, slowing convergence.

ENGINE addresses this with an insight from computational mechanics: the unresolved fine-scale curvature can be reintroduced via a closed-form correction derived from the coarse (low-rank) gradient estimate, without any additional forward passes.

Why Existing Approaches Fall Short

Full-space ZO (MeZO) estimates gradients with random directions in the full parameter space. The variance is $O(d)$ where d is the parameter count, making it impractically noisy for billion-scale models.

Low-rank ZO (LOREN) restricts perturbations to a rank-r subspace. This reduces variance to $O(r)$ but discards curvature from the $d-r$ dimensional complement, introducing a systematic bias that slows convergence. With $r \ll d$, the complement curvature is large.

Increasing the rank r reduces bias but increases memory and noise proportionally. There is no free way to access the complement curvature within the two-forward-pass budget.

Core Mechanism: Variational MultiScale Decomposition

The VMS framework decomposes the parameter space into a coarse subspace $\mathcal{U}$ (spanned by a low-rank basis $U \in \mathbb{R}^{d\times r}$) and its complement $\mathcal{U}^{\perp}$. Any perturbation z decomposes as:

$$z = z_c + z_f, \quad z_c = UU^{\top} z, \quad z_f = (I - UU^{\top})z$$

ENGINE estimates the coarse gradient using the standard two-point ZO estimator restricted to z_c:

$$\hat{g}_c = \frac{\ell(\theta + \mu z_c) - \ell(\theta - \mu z_c)}{2\mu} \cdot z_c$$

using 2 forward passes. The **subgrid correction** δ_f accounts for the fine-scale contribution:

$$\delta_f = (I - UU^{\top}) H_{\text{diag}}\, \hat{g}_c$$

where H_{diag} is a diagonal approximation of the Hessian maintained as an exponential moving average of squared gradient magnitudes. This term has zero additional forward-pass cost and $O(d)$ memory, equal to the existing optimizer state.

The final update is:

$$\theta \leftarrow \theta - \eta\big(\hat{g}_c + \alpha\,\delta_f\big)$$

where α is a correction weighting hyperparameter.

Learning or Inference Procedure

Initialization: Compute U as the top-r right singular vectors of a batch of initial ZO gradient estimates. Maintain H_{diag} as an exponential moving average of $\hat{g}_c^2$.

Per-step update:

1. Sample $z_c \sim \mathcal{N}(0, UU^\top)$ (a low-rank direction).
2. Evaluate $\ell(\theta \pm \mu z_c)$ with 2 forward passes.
3. Compute $\hat{g}_c$ and subgrid correction δ_f.
4. Update θ using $\hat{g}_c + \alpha\,\delta_f$.
5. Update H_{diag} with the new squared gradient components.

Periodic basis refresh: Every $K = 200$ steps, recompute U from recent gradient history via a lightweight SVD of a $d \times K$ matrix of gradient directions.

What the Guarantee Says

Proposition 1 (informal): The expected bias of the ENGINE gradient estimate is strictly lower than that of LOREN with the same rank r, provided $\alpha > 0$ and H_{diag} approximates the diagonal Hessian. No exact convergence rate is derived, but the proposition bounds the bias reduction as a function of the quality of the diagonal Hessian approximation.

The practical speed-up guarantee is empirical, not asymptotic. No formal convergence rate is provided in the paper.

Experimental Findings

- **SuperGLUE (GPT-J-6B):** ENGINE matches or exceeds full-fine-tuning accuracy on 7 of 8 tasks; achieves the same accuracy as LOREN with **2.61× fewer forward passes**.
- **MMLU (Llama-3-8B):** ENGINE achieves 72.3% 5-shot accuracy, vs. 69.1% for LOREN and 70.8% for MeZO at the same forward-pass budget.
- **Alpaca Eval:** ENGINE win-rate vs. LOREN is 61.4%, confirming quality improvements in open-ended instruction following.
- **Memory:** Peak memory overhead of ENGINE vs. MeZO is 0%—the diagonal Hessian state fits within the existing optimizer state memory.

Ablations and Interpretation

Without subgrid correction ($\alpha = 0$): Performance equals LOREN, confirming that the speedup is entirely due to the correction.

Rank r sensitivity: ENGINE with $r = 32$ outperforms LOREN with $r = 128$ on SuperGLUE, demonstrating that the correction compensates for a fourfold reduction in rank.

Hessian approximation quality: Using a richer block-diagonal Hessian estimate improves convergence by 7% but requires 1.3× more memory. The diagonal approximation provides most of the benefit.

Basis refresh interval K: Performance is robust to K in the range 100–500, suggesting that the low-rank basis does not need to be updated frequently.

Reference

Zhuoli Ouyang, Bowen Li, Chen Hao, Yejin Choi. **ENGINE: Endogenous Variational MultiScale Optimization for Zeroth-Order LLM Fine-Tuning.** arXiv:2611.03293. 2026. https://arxiv.org/abs/2611.03293